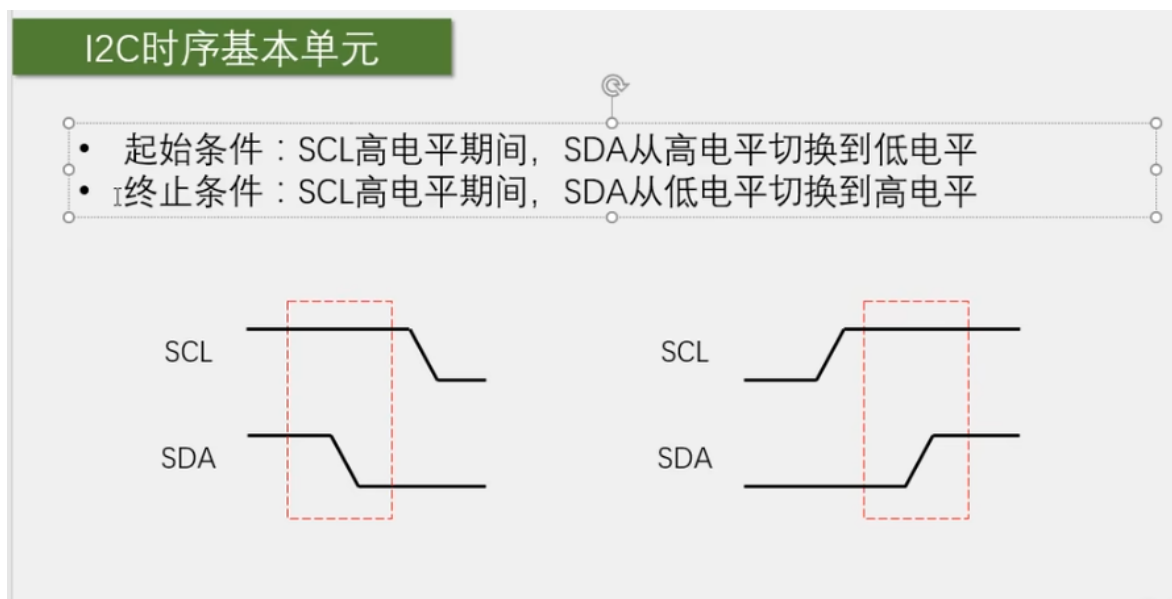


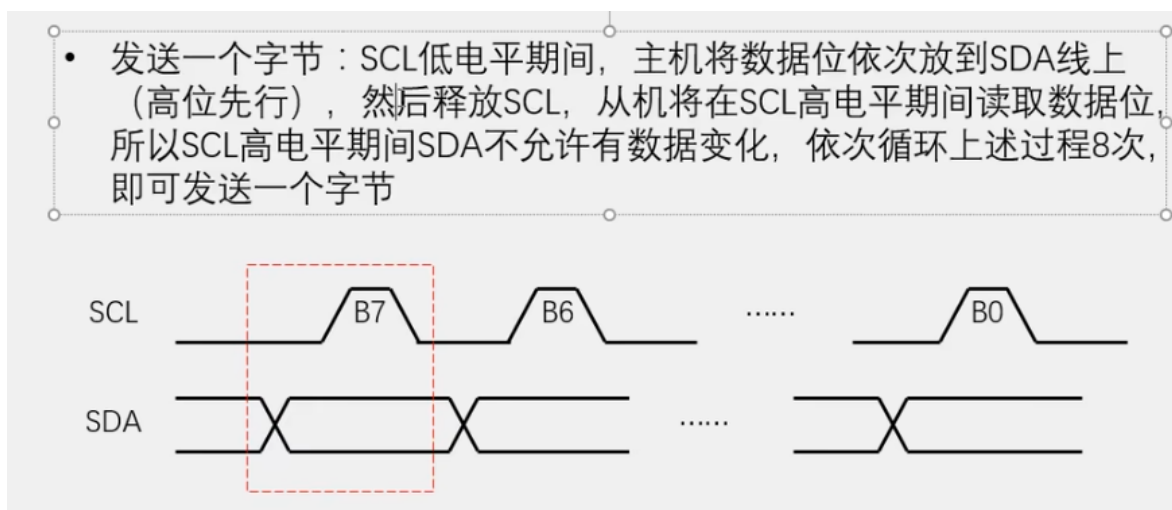
# I2C

开漏输出：上面有上拉电阻 4.7kΩ

推挽输出：上面没有上拉电阻



工作的时候一直是低电平



SDA: 数据

SCL: 正常保持低电平 一旦高电平就开始读

SCL低电平的时候写数据 SCL高电平的时候读数据

(在SCL上升沿开始读)

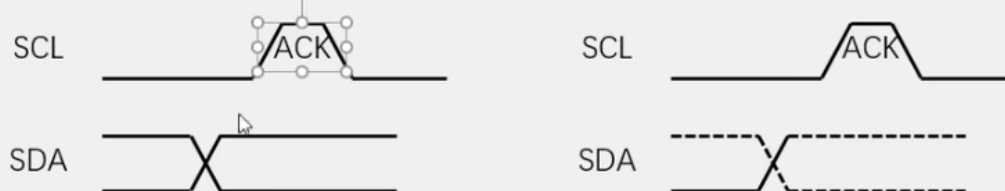
- 接收一个字节：SCL低电平期间，从机将数据位依次放到SDA线上（高位先行），然后释放SCL，主机将在SCL高电平期间读取数据位，所以SCL高电平期间SDA不允许有数据变化，依次循环上述过程8次，即可接收一个字节（主机在接收之前，需要释放SDA）



主机接受前 需要释放SDA

(总线任何一个设备拉低 都是低电平)

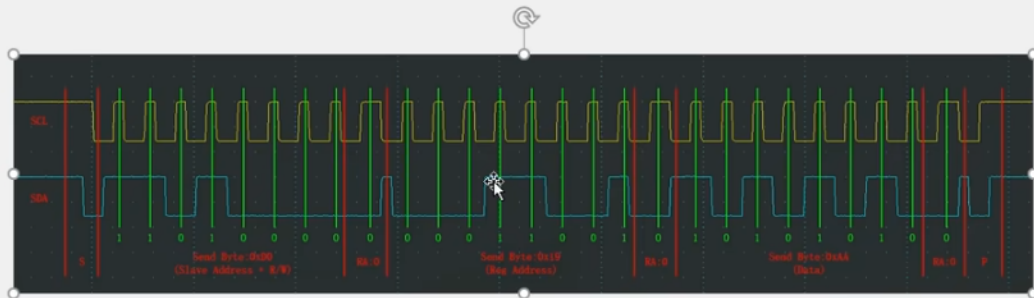
- 发送应答：主机在接收完一个字节之后，在下一个时钟发送一位数据，数据0表示应答，数据1表示非应答
- 接收应答：主机在发送完一个字节之后，在下一个时钟接收一位数据，判断从机是否应答，数据0表示应答，数据1表示非应答（主机在接收之前，需要释放SDA）



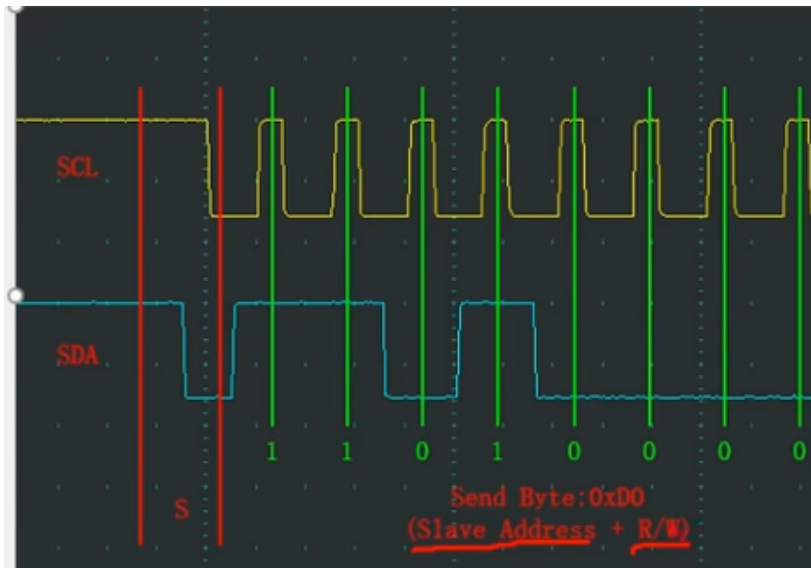
发送和接受应答相当于发送和接受一位数据

## I2C时序

- 指定地址写
- 对于指定设备 (Slave Address), 在指定地址 (Reg Address) 下, 写入指定数据 (Data)



指定设备 (设备ID) + 指定地址 (寄存器地址) + 指定数据 (要写入的数据)



发送从机地址和读写位

(0写 1读)

从机接收后 发送应答位

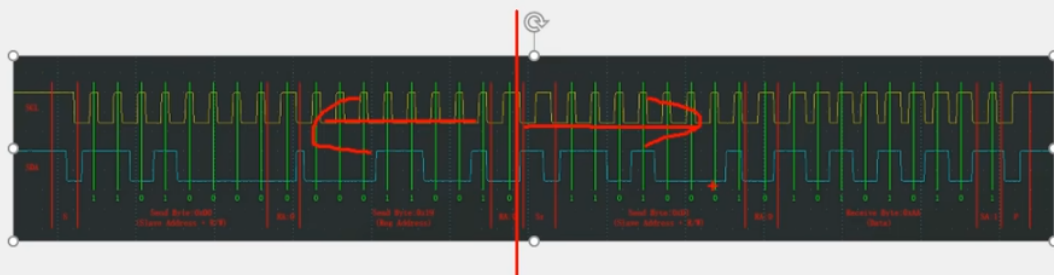
- 当前地址读
- 对于指定设备 (Slave Address)，在当前地址指针指示的地址下，读取从机数据 (Data)



地址指针

写入一个数据或者读出一个数据后 地址指针+1

- 指定地址读
- 对于指定设备 (Slave Address)，在指定地址 (Reg Address) 下，读取从机数据 (Data)



因为我们刚指定了地址

重复起始

读后发送非应答就停止了

软件I2C

两个引脚开漏输出模式 (上拉)

两个引脚都放置高电平 (I2C初始化)

```

void MyI2C_Init(void)
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_OD;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10 | GPIO_Pin_11;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    GPIO_SetBits(GPIOB, GPIO_Pin_10 | GPIO_Pin_11);
}

```

## I2C起始条件

```

void MyI2C_W_SCL(uint8_t BitValue)
{
    GPIO_WriteBit(GPIOB, GPIO_Pin_10, (BitAction)BitValue);
    Delay_us(10);
}

void MyI2C_W_SDA(uint8_t BitValue)
{
    GPIO_WriteBit(GPIOB, GPIO_Pin_11, (BitAction)BitValue);
    Delay_us(10);
}

uint8_t MyI2C_R_SDA(void)
{
    uint8_t BitValue;
    BitValue = GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11);
    Delay_us(10);
    return BitValue;
}

```

为什么要加延时 为了让从机设备跟得上

```

void MyI2C_Start(void)
{
    MyI2C_W_SDA(1);
    MyI2C_W_SCL(1);
    MyI2C_W_SDA(0);
    MyI2C_W_SCL(0);
}

void MyI2C_Stop(void)
{
    MyI2C_W_SDA(0);
    MyI2C_W_SCL(1);
    MyI2C_W_SDA(1);
}

```

```

void MyI2C_SendByte(uint8_t Byte)
{
    uint8_t i;
    for (i = 0; i < 8; i++)
    {
        MyI2C_W_SDA(Byte & (0x80 >> i));
        MyI2C_W_SCL(1);
        MyI2C_W_SCL(0);
    }
}

uint8_t MyI2C_ReceiveByte(void)
{
    uint8_t i, Byte = 0x00;
    MyI2C_W_SDA(1);
    for (i = 0; i < 8; i++)
    {
        MyI2C_W_SCL(1);
        if (MyI2C_R_SDA() == 1){Byte |= (0x80 >> i);}
        MyI2C_W_SCL(0);
    }
    return Byte;
}

void MyI2C_SendAck(uint8_t AckBit)
{
    MyI2C_W_SDA(AckBit);
    MyI2C_W_SCL(1);
    MyI2C_W_SCL(0);
}

uint8_t MyI2C_ReceiveAck(void)
{
    uint8_t AckBit;
    MyI2C_W_SDA(1);
    MyI2C_W_SCL(1);
    AckBit = MyI2C_R_SDA();
    MyI2C_W_SCL(0);
    return AckBit;
}

```

### 对固定操作拼接时序

```

void MPU6050_WriteReg(uint8_t RegAddress, uint8_t Data)
{
    MyI2C_Start();
    MyI2C_SendByte(MPU6050_ADDRESS);
    MyI2C_ReceiveAck();
    MyI2C_SendByte(RegAddress);
    MyI2C_ReceiveAck();
    MyI2C_SendByte(Data);
    MyI2C_ReceiveAck();
    MyI2C_Stop();
}

```

```

uint8_t MPU6050_ReadReg(uint8_t RegAddress)
{
    uint8_t Data;

    MyI2C_Start();
    MyI2C_SendByte(MPU6050_ADDRESS);
    MyI2C_ReceiveAck();
    MyI2C_SendByte(RegAddress);
    MyI2C_ReceiveAck();

    MyI2C_Start();
    MyI2C_SendByte(MPU6050_ADDRESS | 0x01);
    MyI2C_ReceiveAck();
    Data = MyI2C_ReceiveByte();
    MyI2C_SendAck(1);
    MyI2C_Stop();

    return Data;
}

```

一定注意先点名

```

#define MPU6050_ACCEL_XOUT_H 0x3B
#define MPU6050_ACCEL_XOUT_L 0x3C
#define MPU6050_ACCEL_YOUT_H 0x3D
#define MPU6050_ACCEL_YOUT_L 0x3E
#define MPU6050_ACCEL_ZOUT_H 0x3F
#define MPU6050_ACCEL_ZOUT_L 0x40
#define MPU6050_TEMP_OUT_H 0x41
#define MPU6050_TEMP_OUT_L 0x42
#define MPU6050_GYRO_XOUT_H 0x43
#define MPU6050_GYRO_XOUT_L 0x44
#define MPU6050_GYRO_YOUT_H 0x45
#define MPU6050_GYRO_YOUT_L 0x46
#define MPU6050_GYRO_ZOUT_H 0x47
#define MPU6050_GYRO_ZOUT_L 0x48

#define MPU6050_PWR_MGMT_1 0x6B
#define MPU6050_PWR_MGMT_2 0x6C
#define MPU6050_WHO_AM_I 0x75

```

宏定义寄存器

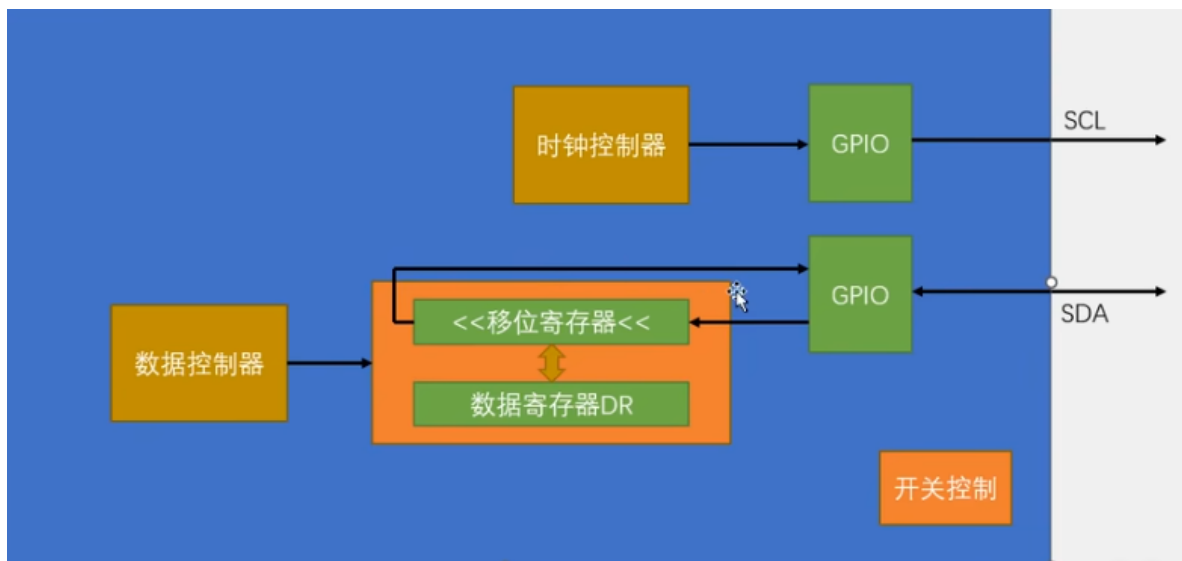
多返回值函数的方法：定义多个全局变量 用指针传递

开始学习硬件I2C：

注意连线

利用I2C外设替换软件I2C代码





复用：控制权交给外设

```
GPIO_InitTypeDef GPIO_InitStructure;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_OD;
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10 | GPIO_Pin_11;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOB, &GPIO_InitStructure);

I2C_InitTypeDef I2C_InitStructure;
I2C_InitStructure.I2C_Mode = I2C_Mode_I2C;
I2C_InitStructure.I2C_ClockSpeed = 50000;
I2C_InitStructure.I2C_DutyCycle = I2C_DutyCycle_2;
I2C_InitStructure.I2C_Ack = I2C_Ack_Enable;
I2C_InitStructure.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7bit;
I2C_InitStructure.I2C_OwnAddress1 = 0x00;
I2C_Init(I2C2, &I2C_InitStructure);
```

```
I2C_Cmd(I2C2, ENABLE);
```

硬件I2C函数 非阻塞式 需要等待标志位

```
void MPU6050_WriteReg(uint8_t RegAddress, uint8_t Data)
{
    I2C_GenerateSTART(I2C2, ENABLE);
    while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_MODE_SELECT) != SUCCESS);

    I2C_Send7bitAddress(I2C2, MPU6050_ADDRESS, I2C_Direction_Transmitter);
    while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED) != SUCCESS);

    I2C_SendData(I2C2, RegAddress);
    while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_BYTE_TRANSMITTING) != SUCCESS);

    I2C_SendData(I2C2, Data);
    while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_BYTE_TRANSMITTED) != SUCCESS);

    I2C_GenerateSTOP(I2C2, ENABLE);
}
```



```

I2C_Send7bitAddress(I2C2, MPU6050_ADDRESS, I2C_Direction_Transmitter);
while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED) != SUCCESS);

I2C_SendData(I2C2, RegAddress);
while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_BYTE_TRANSMITTED) != SUCCESS);

I2C_GenerateSTART(I2C2, ENABLE);
while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_MODE_SELECT) != SUCCESS);

I2C_Send7bitAddress(I2C2, MPU6050_ADDRESS, I2C_Direction_Receiver);
while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_RECEIVER_MODE_SELECTED) != SUCCESS);

I2C_AcknowledgeConfig(I2C2, DISABLE);
I2C_GenerateSTOP(I2C2, ENABLE);

while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_BYTE_RECEIVED) != SUCCESS);
Data = I2C_ReceiveData(I2C2);

I2C_AcknowledgeConfig(I2C2, ENABLE);

return Data;
}

while (I2C_CheckEvent(I2C2, I2C_EVENT_MASTER_MODE_SELECT) != SUCCESS)
{
    Timeout --;
    if (Timeout == 0)
    {
        return;
    }
}

```